

Consigna — Aplicación CRUD con React, React Router y Firebase

Objetivo

Desarrollar una aplicación web utilizando **React** que permita gestionar información almacenada en **Firebase Firestore**.

El proyecto deberá permitir visualizar, buscar, agregar, modificar y eliminar registros, además de contar con una vista individual para consultar el detalle de cada elemento.

La temática propuesta para el proyecto es una **aplicación de películas favoritas**, pero también existe la posibilidad de realizar un proyecto personal utilizando otra temática.

Opción 1 — Proyecto propuesto: Mis películas favoritas

Deberás desarrollar una aplicación que permita gestionar una colección de películas.

Cada película deberá contar, como mínimo, con:

- Título
- Año de estreno
- Género
- Rating
- Estado de favorita

La aplicación deberá permitir:

- Visualizar todas las películas almacenadas.
- Buscar películas por título.

- Agregar nuevas películas.
- Editar películas existentes.
- Eliminar películas.
- Marcar y desmarcar películas como favoritas.
- Acceder a una página de detalle de una película.
- Mostrar información de carga mientras se realizan operaciones.
- Manejar errores básicos durante las operaciones con Firebase.

Opción 2 — Proyecto personal

También podés desarrollar una aplicación basada en **una temática propia**.

Algunas posibilidades son:

- Videojuegos
- Libros
- Series
- Recetas
- Productos
- Lugares turísticos
- Mascotas
- Canciones
- Cursos
- Eventos
- Colecciones personales

La temática es libre, pero **no alcanza con cambiar los nombres de las variables del proyecto de películas**.

El proyecto personal deberá implementar todas las funcionalidades y requisitos técnicos solicitados en esta consigna.

Por ejemplo, si elegís desarrollar un catálogo de videojuegos, deberás poder crear, editar, eliminar, buscar y marcar como favoritos los videojuegos, además de acceder al detalle de cada uno.

Requisitos obligatorios

1. React

La aplicación deberá estar desarrollada utilizando React.

Deberá existir una correcta separación de responsabilidades mediante componentes.

Como mínimo, deberán utilizarse:

- `useState`
- `useEffect`
- `useMemo`
- `useRef`

Cada Hook deberá utilizarse para resolver una necesidad real de la aplicación.

2. React Router

La aplicación deberá utilizar **React Router** para manejar la navegación.

Deberá contar como mínimo con:

- Una ruta principal para el listado.
- Una ruta para un formulario o sección de login.
- Una ruta dinámica para visualizar el detalle de un elemento.
- Una ruta para páginas inexistentes (**404**).

La ruta de detalle deberá utilizar un parámetro dinámico, por ejemplo:

`/movies/:id`

o, en un proyecto personal:

`/products/:id`

`/games/:id`

`/books/:id`

según la temática elegida.

3. Firebase Firestore

Los datos deberán almacenarse en **Firebase Firestore**.

No se deberá utilizar únicamente un array local o **localStorage** como fuente principal de datos.

La aplicación deberá implementar un CRUD completo:

- **Create:** crear registros.
- **Read:** obtener y mostrar registros.
- **Update:** modificar registros.
- **Delete:** eliminar registros.

Además, deberá existir una operación adicional que permita modificar un estado booleano del registro, como por ejemplo:

- Favorito / no favorito.
- Disponible / no disponible.
- Completado / pendiente.
- Activo / inactivo.

4. Servicios

Las operaciones con Firebase deberán estar separadas de los componentes.

Por ejemplo, se deberá contar con un archivo o módulo de servicios que contenga funciones similares a:

```
obtenerTodos()  
obtenerPorId()  
crear()  
actualizar()  
eliminar()  
cambiarEstado()
```

Los componentes no deberían contener directamente toda la lógica de comunicación con Firestore.

5. Variables de entorno

La configuración de Firebase deberá utilizar **variables de entorno**.

No se deberán colocar directamente en el código los valores sensibles o configurables de Firebase.

Por ejemplo:

```
VITE_FIREBASE_API_KEY  
VITE_FIREBASE_AUTH_DOMAIN  
VITE_FIREBASE_PROJECT_ID  
...
```

6. Formulario

Deberá existir un formulario controlado mediante estados de React.

El formulario deberá permitir:

- Crear un nuevo registro.
- Editar un registro existente.
- Limpiar los campos después de una operación.
- Validar que los datos obligatorios hayan sido ingresados.

El formulario deberá utilizar **useRef** para realizar alguna acción sobre un elemento del formulario, por ejemplo, colocar automáticamente el foco sobre un campo.

7. Validación

Los datos ingresados deberán validarse antes de enviarse a Firebase.

Como mínimo, deberán comprobarse los campos obligatorios.

La validación deberá estar separada en una función o módulo independiente.

8. Búsqueda y filtrado

La aplicación deberá contar con un campo de búsqueda que permita filtrar los registros.

El filtrado deberá realizarse utilizando **useMemo**.

Por ejemplo:

Buscar película → mostrar solamente las películas cuyo título coincida.

En un proyecto personal, la búsqueda deberá adaptarse al contenido elegido.

9. Edición

Cada registro deberá contar con una opción para modificar sus datos.

Al seleccionar la opción de editar:

1. El formulario deberá mostrar los datos actuales.
2. El usuario deberá poder modificarlos.
3. Al guardar, los cambios deberán actualizarse en Firestore.
4. La interfaz deberá reflejar los cambios realizados.

10. Eliminación

Cada registro deberá contar con una opción para eliminarlo.

Antes de eliminarlo deberá solicitarse confirmación al usuario.

Una vez confirmada la operación, el registro deberá eliminarse de Firestore y la interfaz deberá actualizarse.

11. Detalle

Cada registro deberá contar con una opción para acceder a una página de detalle.

La página deberá obtener el `id` desde la URL utilizando React Router y consultar el registro correspondiente en Firebase.

Por ejemplo:

`/movies/abc123`

El detalle deberá mostrar información del registro seleccionado.

12. Estados de carga y errores

La aplicación deberá contemplar los estados producidos durante las operaciones asíncronas.

Por ejemplo:

- Cargando registros.
- Guardando.
- Eliminando.
- Actualizando.
- Error al consultar Firebase.
- Error al guardar información.

No es necesario implementar un sistema complejo de manejo de errores, pero el usuario deberá recibir algún tipo de feedback cuando una operación falle.

Organización del proyecto

Se espera una organización similar a:

```
src/  
├── components/  
├── views/
```

└─ services/
└─ validators/
└─ router/
└─ config/
└─ styles/

La estructura puede modificarse según las necesidades del proyecto, pero deberá existir una separación clara entre:

- Componentes.
- Vistas/páginas.
- Servicios.
- Validaciones.
- Configuración de Firebase.
- Ruteo.

Condición para el proyecto personal

Si elegís desarrollar una temática propia, **todos los requisitos anteriores siguen siendo obligatorios.**

Por ejemplo, un proyecto de recetas podría tener:

Receta

└─ nombre
└─ categoría
└─ tiempo
└─ dificultad
└─ favorita

Y debería permitir:

Crear receta

↓

Listar recetas

↓

Buscar receta

↓

Editar receta



Eliminar receta



Marcar como favorita



Ver detalle

La temática puede cambiar, pero **la complejidad y las funcionalidades requeridas deben mantenerse.**

Entrega

El proyecto deberá entregarse con:

- Código fuente completo.
- Proyecto funcional.
- Configuración de Firebase mediante variables de entorno.
- Base de datos de Firestore funcionando.
- **README.md** con una breve explicación del proyecto.
- Instrucciones para instalar y ejecutar la aplicación.

En el caso de elegir un proyecto personal, el **README** deberá indicar también **qué temática se eligió y cómo se relaciona con cada uno de los requisitos de la consigna.**

Importante

La evaluación no estará basada únicamente en que la aplicación "funcione". Se tendrá en cuenta también la **utilización correcta de React, Hooks, React Router, Firebase, servicios, componentes, validaciones y organización del código.**